

User Manual for Nextflow Pipeline Tool

Introduction

Antimicrobial resistance (AMR) is one of the top global health threats, as stated by the World Health Organization (WHO). It leads to resistance against antibiotics, making infections harder to treat. *Pseudomonas aeruginosa* is a major concern in AMR due to its resistance, which makes treating infections such as bloodstream infections and respiratory system infections more challenging.

To address this issue, there is a need for a variant database that aids in clinical studies, and therapeutic research. This tool is designed to create a variant database using FASTA, FASTQ, and metadata CSV files. It generates VCF, CSV, and database (DB) files as output and provides a web interface for analysis.

Workflow

The tool follows these key processes:

- Indexing
- Trimming and quality check
- Alignment and sorting
- Duplicate marking
- Variant calling and annotation
- Variant filtering
- Database creation using SQLite3

Database Structure

The generated database contains:

1. Variant Table with information such as:

- ☐ Chromosome name
 - ☐ Strain name (of the reference)
 - ☐ Genome ID
 - ☐ Amino acid change
 - ☐ Reference allele
 - ☐ Alternative allele
 - ☐ Gene name etc.,
2. Strain Table with fields including:
- ☐ Genome ID
 - ☐ SRA accession
 - ☐ Geographical group
 - ☐ Isolation country
 - ☐ Genome name (strain name)
 - ☐ Taxon ID
 - ☐ Resistant phenotype

This tool enables the creation of an instance variant database for all bacterial species and users must provide the correct input files corresponding to the correct species to ensure accurate results.

Step-by-Step Guide

1. Install Docker.
 2. Use Docker commands to build and run the image.
 3. First, build the database for SnpEff.
 4. Use mkdir to create a new folder, making it easier to run and access system data
 5. Store all files in a single directory, based on the input parameters mentioned below.
 6. Use the Nextflow command with the required parameters to process the data.
-

1. System Requirements:

Minimum Requirements:

- CPU: Dual-core processor
- RAM: 6 GB
- Storage: 8 GB free space
- Python: 3.8 or higher
- Nextflow: Latest version
- Tools: Latest version
- Docker: Latest version
- OS: Windows 10/11, Linux (Ubuntu 20.04+)

Recommended Requirements:

- CPU: Quad-core processor
 - RAM: 14GB or more
 - Storage: 8 GB free space
 - Python: 3.10 or higher
 - Nextflow: Latest version
 - Tools: Latest version
 - Docker: Latest version
 - OS: Windows 10/11, Linux (Ubuntu 22.04+)
-

2. Installation:

A. Installing Nextflow Without

Docker Linux (Ubuntu/Debian-based)

- Install dependencies:
 - `sudo apt update && sudo apt install -y openjdk-21-jdk curl`
- Install Nextflow:

- `curl -s https://get.nextflow.io | bash`
- Move Nextflow to a directory in your system PATH:
 - `sudo mv nextflow /usr/local/bin/`
- Verify installation:
 - `nextflow -v`

B. Installing Required Tools for the Pipeline Without Docker

1. Installing SnpEff(with and without docker for Windows and

Linux) Build snpEff database

1. Download SnpEff (If Docker is already installed, skip this download step and continue with the remaining steps).

Go to the SnpEff download page. Download the latest version of snpeff.zip file and extract it:

```
wget https://snpeff.blob.core.windows.net/versions/snpEff_latest_core.zip
unzip snpEff_latest_core.zip
cd snpEff
```

**** The above wget snpeff link points to an older version. Please use the latest version link available on the official SnpEff download page instead****

Set Up Environment: Ensure SnpEff is executable:

```
chmod +x snpEff/snpEff.jar
cd snpEff
```

```
java -jar snpEff.jar -version -version // This ensures that if snpeff
is already installed.
```

Generating a .bin file from SnpEff for my_bacteria:(Windows and linux with docker)

- Navigate to the SnpEff directory (inside the container): `cd /opt/snpeff/snpEff`
- Check if your species database is available:

```
java -jar snpEff.jar databases | grep "my_bacteria"
```

- Download the database for your species:

```
java -jar snpEff.jar download  
my_bacteria
```

After downloading, SnpEff will automatically generate the required **.bin** file for my_bacteria.(`snpEff/data/my_bacteria/.bin`)

If your species directory is missing files except for .bin, you can copy the entire species folder using the following command:

```
cp -r /opt/snpeff/snpEff/data/Pseudomonas_aeruginosa_pao1 /home/docker/
```

Generating a .bin file from SnpEff for my_bacteria:(Linux without docker)

- Check if your species database is available:

```
java -jar snpEff.jar databases | grep "my_bacteria"
```

- Download the database for your species:

```
java -jar snpEff.jar download  
my_bacteria
```

After downloading, SnpEff will automatically generate the required **.bin** file for my_bacteria.(`snpEff/data/my_bacteria/.bin`)

2. Obtain Genome and Annotation Files

- Download your bacterial genome in FASTA format (.fasta or .fna) from NCBI, Ensembl Bacteria, or another source.
- Obtain the corresponding GFF annotation file (.gff).

3. Use Prokka to Format Files for SnpEff

Format Files for SnpEff:

- Ensure the FASTA file contains a single genome sequence (or properly labeled contigs).
- Verify that the GFF file has accurate annotations, especially for CDS regions.

If annotations are missing or need to be generated, use Prokka: Note :

- This is the preferred installation method from Prokka's official page.
 - **conda install -c conda-forge -c bioconda prokka**

- After installation, verify:
 - **prokka --version**
- Replace my_bacteria with the actual bacterial name
- The output files my_bacteria.fasta and my_bacteria.gff will be used in SnpEff.

Run:

```
prokka --outdir ./prokka_output --prefix my_bacteria my_bacteria.fasta
```

****If the prokka --setupdb command is not working, try running it again. If it still doesn't work, reinstall Prokka.****

4. Configure the SnpEff Database

Locate the snpEff.config file in snpEff directory and open the SnpEff directory
then snpEff.config

file. `nano snpEff.config`

Add a new entry at the end of the file above the Non-standard Databases section:

Note:

- Replace my_bacteria with a unique identifier for your genome.
- Replace MyBacteria with any name.

```
#---
```

```
# Non-standard Databases
```

```
#---
```

```
my_bacteria.genome : MyBacteria
```

5. Set Up the Genome Directory

Inside the data folder in the SnpEff directory, check a subdirectory for your genome:

```
ls snpEff/data/my_bacteria
```

6. Copy the Genome and Annotation Files

Copy the genome FASTA and annotation GFF files into the directory:

```
cp my_bacteria.fasta snpEff/data/my_bacteria/sequences.fna
cp my_bacteria.gff snpEff/data/my_bacteria/genes.gff
```

Use the Prokka output files

- my_bacteria.fasta - sequences.fna
- my_bacteria.gff - genes.gff

7. Build the SnpEff Database

Run the following command to build the database:

```
java -Xmx4g -jar snpEff/snpEff.jar build -gff3 -v my_bacteria
```

8. Handling Errors

If you encounter an error like this:

```
>>>WARNING_FILE_NOT_FOUND: Rare Amino Acid analysis: Cannot read
protein sequence file ' path/to/snpEff/data/au/protein.fa ', nothing done.
ERROR: CDS check file ' path/to/snpEff/data/au/cds.fa ' not
found. ERROR: Protein check file ' path/to/snpEff/data/au/protein.fa ' not
found.
ERROR: Database check failed.
00:00:01 Logging>>>
```

Note : You need to generate cds.fa and protein.fa files.

9. Generate Missing Files

Extract Coding Sequences (CDS)

Use the GFF3 and FASTA files to generate cds.fa.

Note : Take gff file from prokka.

```
gffread genes.gff -g sequences.fna -x cds.fa
```

This extracts CDS sequences and saves them in cds.fa.

Translate CDS to Proteins

Use transeq from the EMBOSS suite to generate protein.fa:

```
transeq -sequence cds.fa -outseq protein.fa
```

This generates protein.fa.

Place the Files in the Correct Directory

Move the generated files to the SnpEff directory:

```
mv cds.fa protein.fa path/to/snpEff/data/au/
```

10. Rebuild the Database

```
java -Xmx4g -jar snpEff/snpEff.jar build -gff3 -v au
```

2. Installing BWA

```
sudo apt update && sudo apt install -y bwa
```

```
Bwa - - help // This ensures that if BWA is already installed.
```

3. Installing Samtools

```
sudo apt update && sudo apt install -y samtools
```

```
Samtools - - help // This ensures that if Samtools is already installed.
```

4. Installing VCFtools

```
sudo apt update && sudo apt install -y vcftools
```

```
Vcftools - - help // This ensures that if VCFtools is already installed.
```


5. Installing BCFtools

```
sudo apt update && sudo apt install -y bcftools
```

```
Bcftools - - help // This ensures that if BCFtools is already installed.
```

6. Installing Fastp

```
conda install -c bioconda fastp
```

```
Fastp - - help // This ensures that if Fastp is already installed.
```

7. Installing Sed, Awk, and Cut

```
sudo apt update && sudo apt install -y sed awk coreutils
```

```
Coreutils - - help // This ensures that if coreutils is already installed.
```

8. Installing Python3

```
sudo apt update && sudo apt install -y python3 python3-pip
```

```
Python3 - - version // This ensures that if Python3 is already installed.
```

9. Installing SQLite

```
sudo apt update && sudo apt install -y sqlite3 || echo "SQLite3 is already installed or an error occurred."
```

```
Sqlite3 - - version // This ensures that if SQLite3 is already installed.
```

3. Installing and Running a Nextflow and Requirement Tools with Docker For

windows: Windows

- Install Docker:
 - Windows: Download and install from [Docker Desktop](#)
 - Login
 - Start Docker
- Open PowerShell or Command Prompt and run:

- Windows Subsystem For linux

- `wsl - - update`

- Go to that Docker file directory and Build Docker:

- `docker build -t my_app .`

(Inside Docker)

- Run Docker:

- `docker run -it -v /:/home/ my_app /bin/sh`

- Inside Docker, change the directory using `cd ..`, then enter the "app" directory.

- `@root:`

- Run the Nextflow pipeline using:

- `nextflow run main.nf {parameters}`

- Example:

```
root@543fcd8eba53:/# nextflow run /home/home/sastra/virudha/files/main.nf
```

```
--ref /home/home/sastra/virudha/files/sequence2.fasta
```

```
--fna_reads "/home/home/sastra/virudha/files/*.fna"
```

```
--snpeff_db 'psedomonaus aeruginosa'
```

```
--chr_name 'NC_00234.12'
```

```
--datadir /home/home/sastra/virudha/files/snpEff
```

```
--main_db /home/home/sastra/virudha/files/meta_final_db.csv
```

```
--output_dir /home/home/sastra/virudha/files
```

```
--new_db /home/home/sastra/virudha/files/files --ref_strain 'pao1'
```

- `/home/` is a mounted directory, so use `/home/` before every file path.

- Example `:(/home/home/sastra/virudha/files/main.nf)`

Or

(Run Nextflow with Docker and Gui support)

- **Install an X Server on Windows**
 - Since Windows does not have an X11 display system like Linux, you need to install an X server to display GUI applications from a Docker container.
- **Install VcXsrv**
- **Download VcXsrv Windows X Server from:**
<https://vcxsrv/>
- **Start VcXsrv and Configure it**
 1. Launch VcXsrv by searching for "XLaunch" in the Windows Start Menu.
 2. Follow these steps in the setup wizard:
 - Choose Multiple Windows, then click Next.
 - Select Start no client, then click Next.
 - Enable "Disable Access Control" (important!) and click Next.
 - Finish the setup - Click Finish to start VcXsrv.

Set the DISPLAY variable:

- Open PowerShell or Command Prompt and run:

```
set DISPLAY=host.docker.internal:0.0
```

Run the container with the proper X11 configuration:

- ```
docker run -e DISPLAY=host.docker.internal:0.0 -v C:\Users\YourUsername\:/home/admin -v C:\Users\YourUsername\Desktop\docker:/app my_app
```

**Note:**

- `docker run` – Starts a new container.
- `-e DISPLAY=host.docker.internal:0.0` – Enables GUI forwarding for Windows using an X server.
- `-v C:\Users\YourUsername\:/home/admin` – Mounts(Windows directory) C:\Users\YourUsername to /home/admin inside the container.
- `-v C:\Users\YourUsername\Desktop\docker:/app` – Mounts (Windows directory) C:\Users\YourUsername\Desktop\docker to /app inside the container.

- `my_app` – Runs the container using the `my_app` image.
- 

## 4. Installing and Running a Nextflow and Requirement Tools with Docker For Linux:

### A. Running the Pipeline Manually

Without Docker in Linux:

- `nextflow run main.nf {parameters} .`

Example Nextflow command:

```
nextflow run /home/sastra/virudha/files/main.nf
--ref /home/sastra/virudha/files/sequence2.fasta
--fna_reads "/sastra/virudha/files/*.fna"
--snpeff_db 'psedomonaus aeruginosa'
--chr_name 'NC_00234.12'
--datadir /home/sastra/virudha/files/snpEff
--main_db /home/sastra/virudha/files/meta_final_db.csv
--output_dir /home/sastra/virudha/files
--new_db /home/sastra/virudha/files/files --ref_strain 'pao1'
```

**Or**

### With Docker in Linux:

- Update the apt package index:
  - `sudo apt-get update`
- Install dependencies required for Docker:
  - `sudo apt-get install \`
  - `ca-certificates \`
  - `curl \`
  - `gnupg \`
  - `lsb-release \`
  - `sudo`
- Add Docker's official GPG key:

- `curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg`
- Set up the stable repository for Docker:
  - `echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null`
- Update the apt package index again:
  - `sudo apt-get update`
- Install Docker Engine:
  - `sudo apt-get install docker-ce docker-ce-cli containerd.io`
- Start and enable Docker:
  - `sudo systemctl start docker`
  - `sudo systemctl enable docker`
- Verify that Docker is installed and running:
  - `sudo docker --version`
- Build Docker:
  - `sudo docker build -t my_app .`

### (Inside Docker Run)

- Run Docker:
  - `sudo docker run -it -v /:/home/ my_app /bin/bash`
- Inside Docker, change the directory using `cd ..`, then enter the "app" directory.
  - `@root:`
- Run the Nextflow pipeline using:
  - `nextflow run main.nf {parameters}`
- Example:

```
root@543fcd8eba53:/# nextflow run /home/home/sastra/virudha/files/main.nf
--ref /home/home/sastra/virudha/files/sequence2.fasta
--fna_reads "/home/home/sastra/virudha/files/*.fna"
```

```
--snpeff_db 'psedomonaus aerginosa'
--chr_name 'NC_00234.12'
--datadir /home/home/sastra/virudha/files/snpEff
--main_db /home/home/sastra/virudha/files/meta_final_db.csv
--output_dir /home/home/sastra/virudha/files
--new_db /home/home/sastra/virudha/files/files --ref_strain 'pao1'
```

- /home/ is a mounted directory, so use /home/ before every file path.
- Example :(/home/home/sastra/virudha/files/main.nf)

**Or**

### **(GUI based)**

Running the pipeline using GUI gives name,path,files and run nextflow.(if python 3 available).

- The Docker folder must contain the gui.py file, and it should be named exactly as gui.py.
- Navigate to the Docker folder - cd /path/to/docker
- Enable X11 access: xhost +
- Run the Docker container:
- `sudo docker run -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix -v /home/admin:/home/admin -v /home/admin/Desktop/docker:/app my_app`
- If any generated file appears locked or cannot be opened, it is likely stored with root ownership. To fix this, change the ownership to your local user.

#### **Check File Ownership:**

- `ls -l /home/path/to/finalbn_database.db`
- If the file is owned by root,then

#### **Change Ownership:**

- `sudo chown sastra:sastra /home/path/to/finalbn_database.db`
- Now, the file should be accessible on your system.
- sastra - local system user.

#### **Note :**

- `-e DISPLAY=$DISPLAY` - Passes the display environment variable for GUI .
- `-v /tmp/.X11-unix:/tmp/.X11-unix` - Shares the X11 socket for graphical output.

- `-v /home/admin:/home/admin` - Mounts your home directory inside the container.
- `-v /home/admin/Desktop/docker:/app` - Mounts your project directory inside the container which contains the `gui.py` file.
- `my_app` - The name of your Docker image.

### Example GUI:

Nextflow Pipeline GUI

Nextflow Script:

/home/virudhagiri/Desktop/ngs/exam.nf

Browse

Reference File:

/home/virudhagiri/Desktop/ngs/gui\_test/sequence2.fasta

Browse

Reference Strain:

pao1

Data Directory:

/home/virudhagiri/Desktop/ngs/gui\_test/snpEff

Browse

FNA Read:

/home/virudhagiri/Desktop/ngs/gui\_test/\*.fna

Browse

Single FASTQ Read:

/home/virudhagiri/Desktop/ngs/gui\_test/\*.fastq

Browse

Forward Read1:

/home/virudhagiri/Desktop/ngs/gui\_test/SRR31882643\_1.fastq

Browse

Reverse Read2:

/home/virudhagiri/Desktop/ngs/gui\_test/SRR31882643\_2.fastq

Browse

Paired Reads:

/home/virudhagiri/Desktop/ngs/gui\_test/\*\_{1,2}.fastq

Browse

Main Database:

/home/virudhagiri/Desktop/database/meta\_asia.csv

Browse

Output Directory:

/home/virudhagiri/Desktop/ngs/gui\_test

Browse

Use Existing Database?

☐ Yes ☒ No

Existing Database Path:

Browse

New Database Path:

/home/virudhagiri/Desktop/ngs/gui\_test/csvfy

Browse

Chromosome Name:

NC\_002516.2

SnpEff Database:

Pseudomonas\_aeruginosa\_pao1

Additional Command:

Run Nextflow

Execution Log:

```

4Glu
4 SRR31882643 NaN NC_002516.2 ... 59/90 pao1 p.Leu5
9Gln

[5 rows x 24 columns]
Inserted variant data for: SRR31882643
Changes committed to the database.
Database connection closed.

Completed at: 16-Mar-2025 19:59:37
Duration : 3m 7s
CPU hours : 0.1
Succeeded : 16

```

## B. Input Parameters

```
: nextflow run path/to/main.nf
```

- **-ref** - Provide the reference genome file for a species (.fasta or .fna).**(eg., path/to/sequence.fna or fasta)**
- **-datadir** - Specify the path to the SnpEff directory. **(eg., path/to/snpEff)**
- **-fna\_read** - Provide assembled genome files (.fna) or the path containing .fna files.  
**eg., Give “path/to/\*.fna”**
- **-read** - Provide raw sequencing reads (.fastq) or the path containing .fastq files.(Single end fastq)  
**eg., Give “ path/to/\*.fastq ”**
- **-read1** - Provide the forward raw read file (.fastq) with \_1 as a delimiter  
**(e.g., path/to/SRR12333\_1.fastq).**
- **-read2** - Provide the reverse raw read file (.fastq) with \_2 as a delimiter  
**(e.g., path/to/SRR12333\_2.fastq).**
- **-reads** - Provide the path containing multiple .fastq files.(Paired end fastq) eg., Give **“path/to/\*\_{1,2}.fastq ”**
- **-main\_db** - Provide a CSV metadata file containing details of assembled genome .fna files or raw reads.**(eg., path/to/meta.csv)**
- **- ref\_strain** - Provide the name of the reference strain as it appears in the FASTA file.  
**(eg., ‘pao1’ )**
- **-output\_dir** - Specify the path where output CSV and VCF files will be stored. This should be a new directory with nothing inside it. **(e.g., path/to/any directory)**
- **-chr\_name** - Provide the name of the chromosome as it appears in the FASTA file.  
**(eg., ‘NC\_257237’)**
- **-snpEff\_db** - Provide the name of the SnpEff database directory.**(eg., ‘psedo\_au’)**
- **-exist\_db** - Provide the path and filename of an existing database file (if available).  
**(eg., path/to/finaldata.db)**
- **-new\_db** - If no existing database is available, provide this parameter along with the path where the new database file should be stored. Importantly, the specified directory should not contain any existing database file. **(e.g., path/to/any directory)**



## C. Input Type

**Ref:** A reference FASTA or FNA file.

### **Fna\_read:**

- Can be a single file (e.g., path/to/287.1738.fna).
- Can be multiple files stored in a directory (path/to/\*.fna).
- If using fna\_read, the file extension must be .fna; otherwise, do not use this parameter.

### **Read:**

- Can be a single file (e.g., path/to/SRR89484.fastq or path/to/ERR83938.fastq).
- Can be multiple files stored in a directory (path/to/\*.fastq).
- If using read, the file extension must be .fastq, and the filename must start with SRR or ERR.
- If the filename does not start with SRR or ERR, rename it accordingly or do not use this parameter.

### **Read 1 and Read 2:**

- Can be single files:
  - path/to/SRR89484\_1.fastq and path/to/SRR89484\_2.fastq.
  - path/to/ERR83938\_1.fastq and path/to/ERR83938\_2.fastq.
- If using read, the file extension must be .fastq, and the filename must start with SRR or ERR.
- \_1 and \_2 must be added before the file extension.
- If the filename does not follow this format, rename it accordingly or do not use this parameter.

### **Reads:**

- Can be a single paired-end file (e.g., path/to/SRR89484\_1 and \_2.fastq or path/to/ERR83938\_1 and \_2.fastq .fastq).
- Can be multiple files stored in a directory (path/to/\*\_{1,2}.fastq).

- The files must not be clipped (e.g., each file should contain both forward and reverse reads).
- If using read, the file extension must be .fastq, and the filename must start with SRR or ERR.
- If the filename d
- paooes not follow this format, rename it accordingly or do not use this parameter.

#### **Main\_db:**

- The file extension must be .csv.
- The file must contain the following fields:
  - antibiotic, genome\_id, genome\_name (strain name), sra\_accession, isolation\_country, geographical\_group, taxon\_id, and resistant\_phenotype.
- Other fields are optional.

---

## **5. Visualization**

### **A. Web Interface (If python3 is available and not working in docker environment)**

1. Loads your generated database file (SQLite) to host a local server
2. Runs using Streamlit with the command:

```
streamlit run path/to/app.py path/to/database.db
```

3. Redirects to a web interface where you can provide filtering options for variant data.
4. Displays Variant table and interactive plots for analysis..

#### **Without Docker Installation:(Linux)**

- `pip install streamlit`
- `echo 'export PATH=$HOME/.local/bin:$PATH' >> ~/.bashrc`
- `source ~/.bashrc`
- `streamlit --version`
- `streamlit run path/to/app.py path/to/database.db`

#### **With Docker Run:(Linux)**

```
sudo docker run -it -v /:/home/ -p 8501:8501 my_app streamlit run
/home/path/to/stream_test_sqlite/Home.py /home/path/to/db_index_few_check_all_last1.db
--server.port=8501 --server.address=0.0.0.0
```

**Note:**

- **docker run -it** - Runs the container in interactive mode with a terminal.
- **-v /:/app/** - Mounts the entire host filesystem (/) to /app/ inside the container
- **-p 8501:8501** - Maps port 8501 from the container to the host for Streamlit access.
- **my\_app** - Specifies the Docker image to run (my\_app).
- **streamlit run /app/path/to/Home.py /app/path/to/db\_index\_few\_check\_all\_last1.db**  
- Runs Streamlit with the given script and database.(ensure both files are inside /app/ ,  
which is the mounted directory. )
- **--server.port=8501** - Runs Streamlit on port 8501.
- **--server.address=0.0.0.0** - Makes Streamlit accessible externally.

This will create and host a link to the web interface.

**With Docker Run:(Windows)**

```
docker run -it -v C:\:/home/ -p 8501:8501 my_app streamlit run /home/path/to/Home.py
/home/path/to/db_index_few_check_all_last1.db --server.port=8501 --server.address=0.0.0.0
```

**Note:**

- **docker run -it** - Runs the container in interactive mode with a terminal.
- **-v C:\:/home/** - Mounts the entire C: drive from Windows to /home/ inside the container.
- **-p 8501:8501** - Maps port 8501 from the container to the host, allowing external access to Streamlit.
- **my\_app** - Specifies the Docker image to run (my\_app).
- **streamlit run /home/path/to/Home.py**  
**/home/path/to/db\_index\_few\_check\_all\_last1.db** - Runs Streamlit with the given script

and database.(ensure both files are inside /home/ , which is the mounted directory. )

- **--server.port=8501** - Runs Streamlit on port 8501.
- **--server.address=0.0.0.0** - Makes Streamlit accessible from any device on the network.

This will create and host a link to the web interface.

**Note :** Minimum 16 GB RAM require.(depends on database size)

## **B. Sqlite Browser Software**

- You can use this software or any SQLite-based software to visualize the database, search, and analyze data using SQL queries.
- 

## **6. Troubleshooting**

### **Common Issues**

- Docker installation problem for windows Check whether Docker is running or stopped, then update WSL using `wsl --update`
- GUI problem: Check whether Xvxcsrc is running or not. If not, click XLaunch and run it..(windows)
- Check whether the parameter is provided correctly, including case sensitivity as uppercase letters will also cause an error.
- Nextflow command not found: Ensure Nextflow is in your system PATH or docker environment PATH.
- snpEff command error: The snpEff configuration file is not found, or the data directory is missing.
  - If you encounter this issue, follow the snpEff manual (as mentioned in the Installation section) to properly recreate the snpEff database, ensure the correct

path to the configuration file, and export the path using `.bashrc`. Additionally, set the necessary permissions using `chmod 777` on the `snpEff` folder.

- Java not found: Install OpenJDK using `sudo apt install openjdk-21-jdk`.